Dr. K.V. Subba Reddy Inst. of Technology

AUTONOMOUS

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Anantapuramu.)

Accredited with NAAC A+ & NBA | Recognized under Section 2 (f) and 12B of UGC Act,1956

: www.drkvsrit.ac.in | : drkvsr.principal@gmail.com

ISO 9001 : 2015 & 50001 : 2018 & ISO 14001 : 2015 Certified Institution

Cell : 9704 333 789 / 790, 944 000 6717

MACHINE LEARNING LAB - 23A31401P

III Year/I Semester CSE - Data Science

DEPARTMENT OF CSE – DATA SCIENCE
Dr K V SUBBA REDDY INSTITUTE OF TECHNOLOGY
Opp. Dupadu (RS),N.H-44, Kurnool-518218, Kurnool District, A.P.

Vision of Dr KVSRIT:

To be a Global Leader in imparting Quality Technical Education to produce Competent, Technically Innovative Engineers imbibed with Research Aptitude, Entrepreneurship and Social Responsibility.

Mission of Dr KVSRIT:

1. To Nurture the Students with Fundamental Engineering Knowledge enriched with Technical Skills.
2. To Create Conducive Environment to nurture Innovation and Interdisciplinary Research.
3. To Develop Professionals through Innovative Pedagogy focusing on Individual Growth, Discipline, Integrity, Ethics and Social Responsibility.
4. To Foster Industry-Institution Partnerships Leading to Skill Development and Entrepreneurship.

III – I Data Science
Machine Learning Lab - 23A31401P

Course Objectives:

- To learn about computing central tendency measures and Data preprocessing techniques
- To learn about classification and regression algorithms
- To apply different clustering algorithms for a problem.

Dr. K.V. Subba Reddy Inst. of Technology

AUTONOMOUS

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Anantapuramu.)

Accredited with NAAC A+ & NBA | Recognized under Section 2 (f) and 12B of UGC Act,1956

: www.drkvsrit.ac.in | : drkvsr.principal@gmail.com

ISO 9001 : 2015 & 50001 : 2018 & ISO 14001 : 2015 Certified Institution

Cell : 9704 333 789 / 790, 944 000 6717

Student Name	
Hall Ticket	
Academic Year	

Signature of the Lab Teacher **HoD**

External Examiner

Internal Examiner

NDEX

S. No.	Title of the Program	Marks
1	Python Program to Compute Central Tendency Measures: Mean, Median, Mode Measure of Dispersion: Variance, Standard Deviation.	
2	Python Program for Data Preprocessing a. Attribute Selection b. Handling Missing Values c. Discretization d. Elimination of Outliers	
3.1	Python code to KNN Classification	
3.2	Python code to KNN Regression	
4	Python code to Decision Tree Classification with Hyperparameter Tuning OR Python code to Demonstrate decision tree algorithm for a classification problem and perform parameter tuning for better results	
5	Python code Decision Tree Regression for a Regression Problem	
6	Python code to Apply Random Forest algorithm for classification and regression	
7	Python code to Demonstrate Naïve Bayes Classification algorithm.	
8	Python code to Apply Support Vector algorithm for classification	
9	Python code to Demonstrate simple linear regression algorithm for a regression problem	
10	Python code to Apply Logistic regression algorithm for a classification problem	
11	Python code to Demonstrate the use of Expectation Maximization based clustering algorithm	

Prog – 1

Python Program to Compute Central Tendency Measures: Mean, Median, Mode Measure of Dispersion: Variance, Standard Deviation.

```
import statistics
# Input data
data = [2, 4, 4, 4, 5, 5, 7, 9]
print("Given Data:", data)
# Central Tendency Measures
mean = statistics.mean(data)
median = statistics.median(data)
mode = statistics.mode(data)
# Measure of Dispersion
variance = statistics.variance(data)         # Sample Variance
std_deviation = statistics.stdev(data)       # Sample Standard Deviation
population_variance = statistics.pvariance(data)
population_std = statistics.pstdev(data)
# Display Results
print("\n--- Measures of Central Tendency ---")
print("Mean               :", mean)
print("Median             :", median)
print("Mode               :", mode)
print("\n--- Measures of Dispersion ---")
print("Sample Variance     :", variance)
print("Sample Std Deviation:", std_deviation)
print("Population Variance :", population_variance)
print("Population Std Dev. :", population_std)
```

OUTPUT

```
Given Data: [2, 4, 4, 4, 5, 5, 7, 9]

--- Measures of Central Tendency ---
Mean                : 5
Median              : 4.5
Mode                : 4

--- Measures of Dispersion ---
Sample Variance     : 4.571428571428571
Sample Std Deviation: 2.138089935299395
Population Variance : 4
Population Std Dev. : 2.0
```

Prog-2

Python Program for Data Preprocessing

a. Attribute Selection

b. Handling Missing Values

c. Discretization

d. Elimination of Outliers

==========

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import KBinsDiscretizer
# ----------------------------
# Step 1: Create Sample Dataset
# ----------------------------
data = {
        'Age': [25, 30, np.nan, 45, 28, 60, 35, 100],
        'Salary': [25000, 30000, 40000, np.nan, 28000, 90000, 35000, 500000],
        'Experience': [2, 5, 4, 10, 3, 20, np.nan, 30],
        'Department': ['HR', 'IT', 'IT', 'Finance', 'HR', 'IT', 'Finance', 'HR']
    }
df = pd.DataFrame(data)
print("Original Dataset")
print(df)
# ============================================================
# a. ATTRIBUTE SELECTION
# ============================================================
print("\n==============================")
print("ATTRIBUTE SELECTION")
print("==============================")
selected_attributes = df[['Age', 'Salary', 'Experience']]
print(selected_attributes)
# ============================================================
# b. HANDLING MISSING VALUES
# ============================================================
print("\n==============================")
print("HANDLING MISSING VALUES")
print("==============================")
imputer = SimpleImputer(strategy='mean')
selected_attributes[['Age', 'Salary', 'Experience']] = imputer.fit_transform(
selected_attributes[['Age', 'Salary', 'Experience']])
print(selected_attributes)
# ============================================================
# c. DISCRETIZATION
# ============================================================
print("\n==============================")
```

```
print("DISCRETIZATION")
print("==============================")
discretizer = KBinsDiscretizer(
   n_bins=3,
   encode='ordinal',
   strategy='uniform'
)
selected_attributes['Salary_Bin'] = discretizer.fit_transform( selected_attributes[['Salary']])
print(selected_attributes)
# ============================================================
# d. ELIMINATION OF OUTLIERS (IQR Method)
# ============================================================
print("\n==============================")
print("ELIMINATION OF OUTLIERS")
print("==============================")
Q1 = selected_attributes['Salary'].quantile(0.25)
Q3 = selected_attributes['Salary'].quantile(0.75)
IQR = Q3 - Q1
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR
cleaned_data = selected_attributes[
                                    (selected_attributes['Salary'] >= lower_limit) &
                                    (selected_attributes['Salary'] <= upper_limit)
                                  ]
print("Dataset after removing outliers")
print(cleaned_data)
```

OUTPUT

```
Original Dataset
     Age    Salary  Experience Department
0   25.0   25000.0         2.0         HR
1   30.0   30000.0         5.0         IT
2    NaN   40000.0         4.0         IT
3   45.0       NaN        10.0    Finance
4   28.0   28000.0         3.0         HR
5   60.0   90000.0        20.0         IT
6   35.0   35000.0         NaN    Finance
7  100.0  500000.0        30.0         HR
==============================
ATTRIBUTE SELECTION
==============================
     Age    Salary  Experience
0   25.0   25000.0         2.0
1   30.0   30000.0         5.0
2    NaN   40000.0         4.0
3   45.0       NaN        10.0
4   28.0   28000.0         3.0
5   60.0   90000.0        20.0
6   35.0   35000.0         NaN
7  100.0  500000.0        30.0
```

```
==============================
HANDLING MISSING VALUES
==============================
      Age         Salary       Experience
0   25.000000   25000.000000    2.000000
1   30.000000   30000.000000    5.000000
2   46.142857   40000.000000    4.000000
3   45.000000  106857.142857   10.000000
4   28.000000   28000.000000    3.000000
5   60.000000   90000.000000   20.000000
6   35.000000   35000.000000   10.571429
7  100.000000  500000.000000   30.000000

==============================
DISCRETIZATION
==============================
       Age         Salary       Experience  Salary_Bin
0   25.000000   25000.000000    2.000000         0.0
1   30.000000   30000.000000    5.000000         0.0
2   46.142857   40000.000000    4.000000         0.0
3   45.000000  106857.142857   10.000000         0.0
4   28.000000   28000.000000    3.000000         0.0
5   60.000000   90000.000000   20.000000         0.0
6   35.000000   35000.000000   10.571429         0.0
7  100.000000  500000.000000   30.000000         2.0

==============================
ELIMINATION OF OUTLIERS
==============================
Dataset after removing outliers
       Age         Salary      Experience  Salary_Bin
0  25.000000   25000.000000    2.000000         0.0
1  30.000000   30000.000000    5.000000         0.0
2  46.142857   40000.000000    4.000000         0.0
3  45.000000  106857.142857   10.000000         0.0
4  28.000000   28000.000000    3.000000         0.0
5  60.000000   90000.000000   20.000000         0.0
6  35.000000   35000.000000   10.571429         0.0
```

Prog – 3

3.1. KNN Classification

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
# Create KNN Classifier
knn = KNeighborsClassifier(n_neighbors=3)
# Train the model
knn.fit(X_train, y_train)
# Predict on test data
y_pred = knn.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("\nClassification Report")
print(classification_report(y_test, y_pred))
# Predict a new flower
sample = [[5.1, 3.5, 1.4, 0.2]]
prediction = knn.predict(sample)
print("\nPredicted Class:", iris.target_names[prediction][0])
```

OUTPUT

```
Accuracy: 1.0

Classification Report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45


Predicted Class: setosa
```

3.2 KNN Regression

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, r2_score
# Load California Housing dataset
housing = fetch_california_housing()
X = housing.data
y = housing.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42)
# Create KNN Regressor
knn = KNeighborsRegressor(n_neighbors=5)
# Train model
knn.fit(X_train, y_train)
# Predict
y_pred = knn.predict(X_test)
# Evaluation
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("Mean Squared Error:", mse)
print("R2 Score:", r2)
# Predict for a new sample
sample = [X_test[0]]
prediction = knn.predict(sample)
print("\nPredicted House Price:", prediction[0])
```

OUTPUT

```
Mean Squared Error: 1.1186823858768293
R2 Score: 0.14631049965900345

Predicted House Price: 1.623
```

Prog-4

Python code to Decision Tree Classification with Hyperparameter Tuning

OR

Python code to Demonstrate decision tree algorithm for a classification problem and perform parameter tuning for better results

```
# 1. Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt

# 2. Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

print("Feature Names:")
print(iris.feature_names)
print("\nTarget Classes:")
print(iris.target_names)
print("\nDataset Shape:", X.shape)

# 3. Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(    X,    y,    test_size=0.20,    random_state=42,
    stratify=y )

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# 4. Create the initial Decision Tree model
dt_model = DecisionTreeClassifier(random_state=42)

# 5. Train the model
dt_model.fit(X_train, y_train)

# 6. Predict test data
y_pred = dt_model.predict(X_test)

# 7. Evaluate initial model
initial_accuracy = accuracy_score(y_test, y_pred)

print("\n--- Initial Decision Tree Results ---")
print("Accuracy:", initial_accuracy)
```

```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print( classification_report( y_test, y_pred, target_names=iris.target_names ))

# 8. Define parameters for tuning
param_grid = {
                'criterion': ['gini', 'entropy'],
                'max_depth': [2, 3, 4, 5, None],
                'min_samples_split': [2, 5, 10],
                'min_samples_leaf': [1, 2, 4]
              }

# 9. Perform Grid Search
grid_search = GridSearchCV( estimator=DecisionTreeClassifier(random_state=42),
  param_grid=param_grid, cv=5, scoring='accuracy', n_jobs=-1)

grid_search.fit(X_train, y_train)

# 10. Display best parameters
print("\n--- Parameter Tuning Results ---")
print("Best Parameters:")
print(grid_search.best_params_)

print("\nBest Cross-Validation Accuracy:")
print(grid_search.best_score_)

# 11. Get the best model
best_model = grid_search.best_estimator_

# 12. Make predictions using tuned model
y_pred_tuned = best_model.predict(X_test)

# 13. Evaluate tuned model
tuned_accuracy = accuracy_score(y_test, y_pred_tuned)

print("\n--- Tuned Decision Tree Results ---")
print("Accuracy:", tuned_accuracy)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_tuned))

print("\nClassification Report:")
print(
```

```
        classification_report(
            y_test,
            y_pred_tuned,
            target_names=iris.target_names
        )
    )

# 14. Compare accuracies
print("\n--- Accuracy Comparison ---")
print("Initial Model Accuracy :", initial_accuracy)
print("Tuned Model Accuracy   :", tuned_accuracy)

# 15. Visualize the tuned Decision Tree
plt.figure(figsize=(15, 9))
plot_tree(
            best_model,
            feature_names=iris.feature_names,
            class_names=iris.target_names,
            filled=True,
            rounded=True
        )

plt.title("Tuned Decision Tree Classifier")
plt.show()
```

OUTPUT

```
Feature Names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
'petal width (cm)']

Target Classes: ['setosa' 'versicolor' 'virginica']

Dataset Shape          : (150, 4)

Training samples       : 120
Testing samples        : 30

--- Initial Decision Tree Results ---
Accuracy: 0.9333333333333333

Confusion Matrix:
                 [[10  0  0]
                  [ 0  9  1]
                  [ 0  1  9]]

Classification Report:
               precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.90      0.90      0.90        10
   virginica       0.90      0.90      0.90        10
```

```
    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30


--- Parameter Tuning Results ---
Best Parameters:
{'criterion': 'gini', 'max_depth': 4, 'min_samples_leaf': 1,
'min_samples_split': 2}

Best Cross-Validation Accuracy:0.9416666666666668

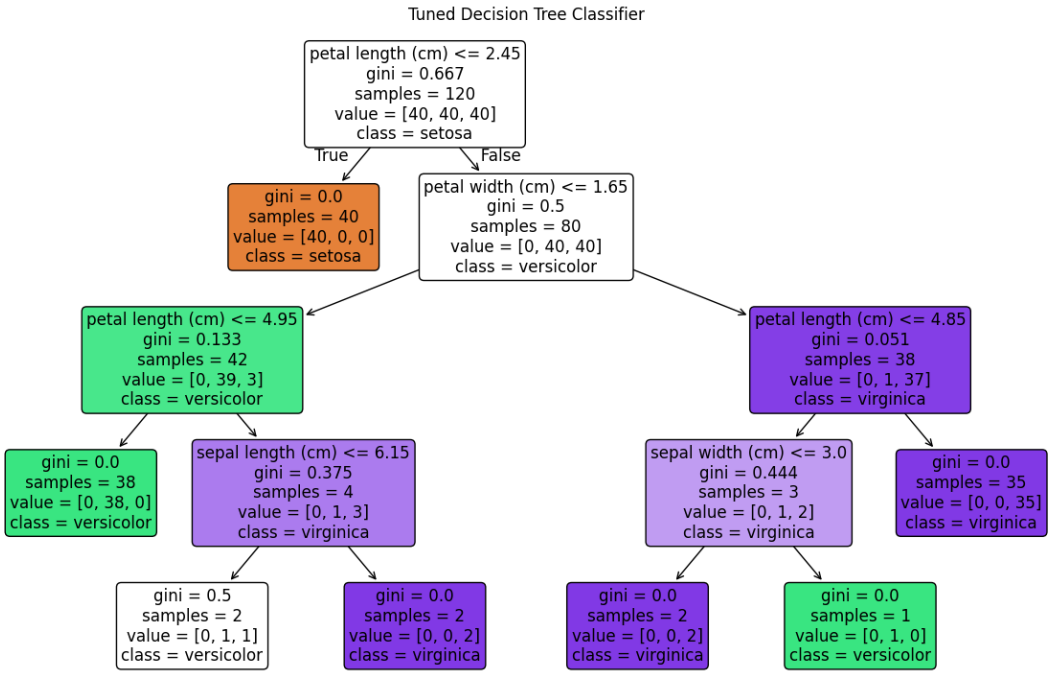
--- Tuned Decision Tree Results ---
Accuracy: 0.9333333333333333

Confusion Matrix:
                [[10  0  0]
                 [ 0  9  1]
                 [ 0  1  9]]

Classification Report:
                 precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.90      0.90      0.90        10
   virginica       0.90      0.90      0.90        10
    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30

--- Accuracy Comparison ---
Initial Model Accuracy : 0.9333333333333333
Tuned Model Accuracy   : 0.9333333333333333
```



Prog-5

Python code Decision Tree Regression for a Regression Problem

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# ------------------------------------------------------
# Step 1: Create Sample Dataset
# ------------------------------------------------------
# Input feature: Hours studied
X = np.array([
                [11], [12], [13], [14], [15],
                [1], [2], [3], [4], [5],
                [6], [7], [8], [9], [10]
            ])

# Target: Marks obtained
y = np.array([
                82, 85, 88, 92, 95,
                35, 40, 45, 50, 55,
                60, 65, 70, 75, 80
            ])
# ------------------------------------------------------
# Step 2: Split Dataset
# ------------------------------------------------------

X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.20, random_state=42)

print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))

# ------------------------------------------------------
# Step 3: Create Decision Tree Regressor
# ------------------------------------------------------

model = DecisionTreeRegressor(    max_depth=4,    random_state=42)

# ------------------------------------------------------
# Step 4: Train the Model
# ------------------------------------------------------

model.fit(X_train, y_train)

print("\nDecision Tree Regression model trained successfully.")
```

```
# ------------------------------------------------------
# Step 5: Predict Test Data
# ------------------------------------------------------

y_pred = model.predict(X_test)

print("\nActual vs Predicted Values")
print("--------------------------------")

for actual, predicted in zip(y_test, y_pred):
    print(f"Actual = {actual:5.2f}    Predicted = {predicted:5.2f}")

# ------------------------------------------------------
# Step 6: Calculate Evaluation Metrics
# ------------------------------------------------------

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nPerformance Evaluation")
print("--------------------------------")
print(f"MAE  : {mae:.2f}")
print(f"MSE  : {mse:.2f}")
print(f"RMSE : {rmse:.2f}")
print(f"R²   : {r2:.4f}")

# ------------------------------------------------------
# Step 7: Predict New Values
# ------------------------------------------------------

new_data = np.array([    [5],    [10],    [12]])

new_predictions = model.predict(new_data)

print("\nPredictions for New Data")
print("--------------------------------")

for hours, prediction in zip(new_data.flatten(), new_predictions):
    print(
                f"Hours Studied = {hours:2d} "
                f"--> Predicted Marks = {prediction:.2f}"
          )
```

OUT PUT

```
Training samples: 12
Testing samples : 3

Decision Tree Regression model trained successfully.

Actual vs Predicted Values
--------------------------------
Actual = 55.00   Predicted = 50.00
Actual = 65.00   Predicted = 60.00
Actual = 82.00   Predicted = 80.00

Performance Evaluation
--------------------------------
MAE  : 4.00
MSE  : 18.00
RMSE : 4.24
R²   : 0.8551

Predictions for New Data
--------------------------------
Hours Studied =  5 --> Predicted Marks = 50.00
Hours Studied = 10 --> Predicted Marks = 80.00
Hours Studied = 12 --> Predicted Marks = 85.00
```

Prog-6

Python code to Apply Random Forest algorithm for classification and regression

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import ( accuracy_score, classification_report, confusion_matrix)
# 1. Load dataset
iris = load_iris()
X = iris.data
y = iris.target
print("Dataset shape:", X.shape)
print("Classes:", iris.target_names)
# 2. Split dataset
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.20, random_state=42,
   stratify=y)
# 3. Create Random Forest classifier
model = RandomForestClassifier(
                                    n_estimators=100,
                                    max_depth=None,
                                    random_state=42
                                )
# 4. Train model
model.fit(X_train, y_train)
# 5. Make predictions
y_pred = model.predict(X_test)
# 6. Evaluate model
accuracy = accuracy_score(y_test, y_pred)
print("\n========== RANDOM FOREST CLASSIFICATION ==========")
print("Accuracy:", round(accuracy, 4))
print("\nClassification Report:")
print(classification_report( y_test, y_pred, target_names=iris.target_names))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
# 7. Feature importance
print("\nFeature Importance:")
for feature, importance in zip( iris.feature_names, model.feature_importances_):
   print(f"{feature}: {importance:.4f}")
# 8. Predict a new sample
sample = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(sample)
print("\nNew Sample Prediction:", iris.target_names[prediction[0]])
```

OUTPUT

```
Dataset shape: (150, 4)
Classes: ['setosa' 'versicolor' 'virginica']

========== RANDOM FOREST CLASSIFICATION ==========
Accuracy: 0.9

Classification Report:
              precision    recall  f1-score   support
      setosa       1.00      1.00      1.00        10
  versicolor       0.82      0.90      0.86        10
   virginica       0.89      0.80      0.84        10
    accuracy                           0.90        30
   macro avg       0.90      0.90      0.90        30
weighted avg       0.90      0.90      0.90        30

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  2  8]]

Feature Importance:
sepal length (cm)      : 0.1163
sepal width (cm)       : 0.0150
petal length (cm)      : 0.4315
petal width (cm)       : 0.4372

New Sample Prediction: setosa
```

Prog-7

Python code to Demonstrate Naïve Bayes Classification algorithm.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import ( accuracy_score, precision_score, recall_score,
  f1_score, classification_report, confusion_matrix)
# ------------------------------------------------
# 1. Load the Iris Dataset
# ------------------------------------------------
iris = load_iris()

X = iris.data
y = iris.target
print("Dataset Shape:", X.shape)
print("Feature Names:", iris.feature_names)
print("Class Names:", iris.target_names)
# ------------------------------------------------
# 2. Split Dataset into Training and Testing Sets
# ------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.20, random_state=42,
  stratify=y)
print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))
# ------------------------------------------------
# 3. Create Naive Bayes Model
# ------------------------------------------------
model = GaussianNB()
# ------------------------------------------------
# 4. Train the Model
# ------------------------------------------------
model.fit(X_train, y_train)
# ------------------------------------------------
# 5. Make Predictions
# ------------------------------------------------
y_pred = model.predict(X_test)
# ------------------------------------------------
# 6. Evaluate the Model
# ------------------------------------------------
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score( y_test, y_pred, average="weighted", zero_division=0)
recall = recall_score(
                        y_test,
                        y_pred,
                        average="weighted",
```

```
                        zero_division=0
                    )
f1 = f1_score(
                    y_test,
                    y_pred,
                    average="weighted",
                    zero_division=0
                )
print("\n========== NAIVE BAYES CLASSIFICATION ==========")
print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-Score :", round(f1, 4))
# ------------------------------------------------
# 7. Classification Report
# ------------------------------------------------
print("\nClassification Report:")
print(    classification_report( y_test, y_pred, target_names=iris.target_names, zero_division=0 ))
# ------------------------------------------------
# 8. Confusion Matrix
# ------------------------------------------------
print("Confusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)
# ------------------------------------------------
# 9. Predict a New Sample
# ------------------------------------------------
new_sample = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(new_sample)
print("\nNew Sample:", new_sample)
print(    "Predicted Class:",    iris.target_names[prediction[0]])
# ------------------------------------------------
# 10. Prediction Probabilities
# ------------------------------------------------
probabilities = model.predict_proba(new_sample)
print("\nPrediction Probabilities:")
for class_name, probability in zip(    iris.target_names,    probabilities[0]):
    print(        f"{class_name}: {probability:.4f}"    )
```

OUTPUT

```
Dataset Shape: (150, 4)
Feature Names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
'petal width (cm)']
Class Names: ['setosa' 'versicolor' 'virginica']

Training Samples: 120
Testing Samples: 30

========== NAIVE BAYES CLASSIFICATION ==========
Accuracy : 0.9667
Precision: 0.9697
Recall   : 0.9667
F1-Score : 0.9666

Classification Report:
               precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.90      0.95        10
   virginica       0.91      1.00      0.95        10
    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30

Confusion Matrix:
                [[10  0  0]
                 [ 0  9  1]
                 [ 0  0 10]]

New Sample: [[5.1, 3.5, 1.4, 0.2]]
Predicted Class: setosa

Prediction Probabilities:
setosa: 1.0000
versicolor: 0.0000
virginica: 0.0000
```

Prog-8

Python code to Apply Support Vector algorithm for classification

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import ( accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix)
# ------------------------------------------------
# 1. Load Dataset
# ------------------------------------------------
iris = load_iris()
X = iris.data
y = iris.target
print("Dataset Shape:", X.shape)
print("Feature Names:", iris.feature_names)
print("Class Names:", iris.target_names)
# ------------------------------------------------
# 2. Split Dataset
# ------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.20, random_state=42,
    stratify=y)
print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))
# ------------------------------------------------
# 3. Feature Scaling
# ------------------------------------------------
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# ------------------------------------------------
# 4. Create SVM Classifier
# ------------------------------------------------
model = SVC( kernel="rbf", C=1.0, gamma="scale")
# ------------------------------------------------
# 5. Train the Model
# ------------------------------------------------
model.fit(X_train_scaled, y_train)
# ------------------------------------------------
# 6. Make Predictions
# ------------------------------------------------
y_pred = model.predict(X_test_scaled)
# ------------------------------------------------
# 7. Evaluate Model
# ------------------------------------------------
```

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score( y_test, y_pred, average="weighted", zero_division=0)
recall = recall_score( y_test, y_pred, average="weighted", zero_division=0)
f1 = f1_score( y_test, y_pred, average="weighted", zero_division=0)
print("\n========== SVM CLASSIFICATION ==========")
print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-Score :", round(f1, 4))
# ------------------------------------------------
# 8. Classification Report
# ------------------------------------------------
print("\nClassification Report:")
print( classification_report( y_test, y_pred, target_names=iris.target_names,
      zero_division=0 ))
# ------------------------------------------------
# 9. Confusion Matrix
# ------------------------------------------------
print("Confusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)
# ------------------------------------------------
# 10. Predict a New Sample
# ------------------------------------------------
new_sample = [[5.1, 3.5, 1.4, 0.2]]
new_sample_scaled = scaler.transform(new_sample)
prediction = model.predict(new_sample_scaled)
print("\nNew Sample:", new_sample)
print( "Predicted Class:", iris.target_names[prediction[0]])
```

OUTPUT

```
Dataset Shape: (150, 4)
Feature Names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
'petal width (cm)']
Class Names: ['setosa' 'versicolor' 'virginica']

Training Samples: 120
Testing Samples: 30

========== SVM CLASSIFICATION ==========
Accuracy : 0.9667
Precision: 0.9697
Recall   : 0.9667
F1-Score : 0.9666

Classification Report:
               precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.90      0.95        10
   virginica       0.91      1.00      0.95        10
    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30

Confusion Matrix:
        [[10  0  0]
         [ 0  9  1]
         [ 0  0 10]]

New Sample: [[5.1, 3.5, 1.4, 0.2]]
Predicted Class: setosa
```

Prog-9

Python code to Demonstrate simple linear regression algorithm for a regression problem

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
# 1. Input data
# X = Independent variable
# Y = Dependent variable
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
Y = np.array([3, 4, 5, 7, 8, 9, 10, 12, 13, 15])

# 2. Create the Linear Regression model
model = LinearRegression()

# 3. Train the model
model.fit(X, Y)

# 4. Predict values
Y_pred = model.predict(X)

# 5. Display model parameters
print("Simple Linear Regression Results")
print("--------------------------------")
print("Slope (Coefficient):", model.coef_[0])
print("Intercept:", model.intercept_)

# 6. Display predictions
print("\nActual vs Predicted Values")
print("--------------------------------")
for actual, predicted in zip(Y, Y_pred):
    print(f"Actual: {actual:5.2f}   Predicted: {predicted:5.2f}")

# 7. Evaluate the model
mse = mean_squared_error(Y, Y_pred)
r2 = r2_score(Y, Y_pred)

print("\nModel Evaluation")
print("--------------------------------")
print("Mean Squared Error (MSE):", mse)
print("R-squared (R²):", r2)

# 8. Predict a new value
new_X = np.array([[11]])
new_prediction = model.predict(new_X)
```

```
print("\nPrediction for X = 11")
print("-------------------------------")
print("Predicted Y:", new_prediction[0])

# 9. Plot the regression line
plt.scatter(X, Y, label="Actual Data")
plt.plot(X, Y_pred, label="Regression Line")
plt.xlabel("Independent Variable (X)")
plt.ylabel("Dependent Variable (Y)")
plt.title("Simple Linear Regression")
plt.legend()
plt.grid(True)
plt.show()
```

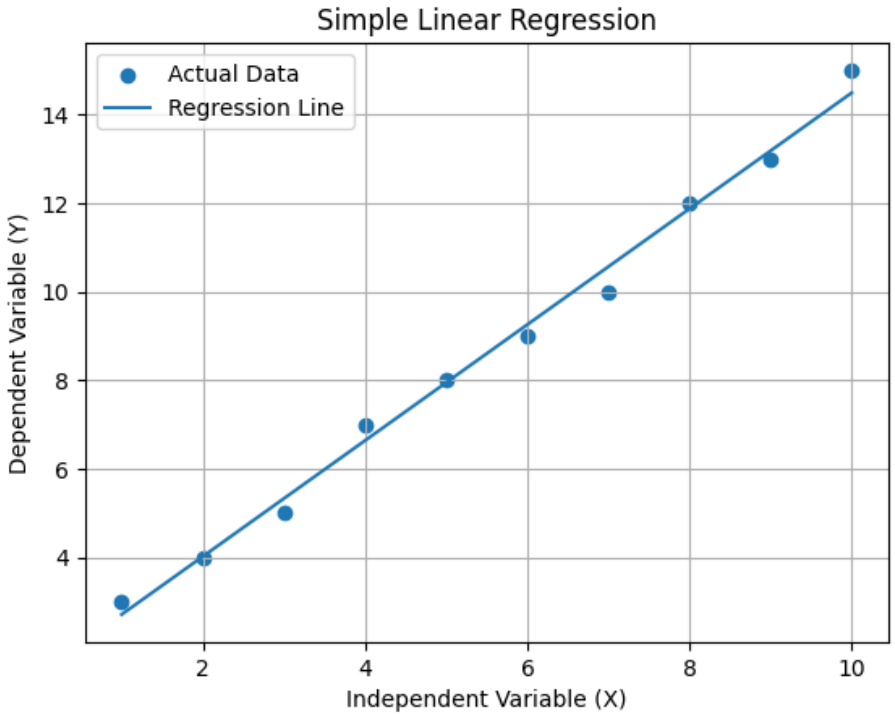
OUTPUT

```
Simple Linear Regression Results
--------------------------------
Slope (Coefficient): 1.309090909090909
Intercept: 1.3999999999999995

Actual vs Predicted Values
--------------------------------
Actual:  3.00   Predicted:  2.71
Actual:  4.00   Predicted:  4.02
Actual:  5.00   Predicted:  5.33
Actual:  7.00   Predicted:  6.64
Actual:  8.00   Predicted:  7.95
Actual:  9.00   Predicted:  9.25
Actual: 10.00   Predicted: 10.56
Actual: 12.00   Predicted: 11.87
Actual: 13.00   Predicted: 13.18
Actual: 15.00   Predicted: 14.49

Model Evaluation
--------------------------------
Mean Squared Error (MSE): 0.10181818181818189
R-squared (R²): 0.992849846782431

Prediction for X = 11
--------------------------------
Predicted Y: 15.8
```



Prog-10

Python code to Apply Logistic regression algorithm for a classification problem

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
                                accuracy_score,
                                confusion_matrix,
                                classification_report
                            )
# ------------------------------------------------
# 1. Create sample dataset
# ------------------------------------------------
# X = Feature (e.g., study hours)
# y = Target class
# 0 = Fail
# 1 = Pass
X = np.array([
                    [1], [2], [3], [4], [5],
                    [6], [7], [8], [9], [10],
                    [11], [12], [13], [14], [15]
                ])
y = np.array([
                0, 0, 0, 0, 0,
                0, 1, 1, 1, 1,
                1, 1, 1, 1, 1
            ])
# ------------------------------------------------
# 2. Split data into training and testing sets
# ------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
                                                        X, y,
                                                        test_size=0.30,
                                                        random_state=42,
                                                        stratify=y
                                                    )
# ------------------------------------------------
# 3. Create Logistic Regression model
# ------------------------------------------------
model = LogisticRegression()
# ------------------------------------------------
# 4. Train the model
# ------------------------------------------------
model.fit(X_train, y_train)
```

```
# ------------------------------------------------
# 5. Make predictions
# ------------------------------------------------
y_pred = model.predict(X_test)
# ------------------------------------------------
# 6. Display actual and predicted values
# ------------------------------------------------
print("Actual vs Predicted Values")
print("-------------------------------")
for actual, predicted in zip(y_test, y_pred):
    print(f"Actual: {actual}    Predicted: {predicted}")
# ------------------------------------------------
# 7. Calculate accuracy
# ------------------------------------------------
accuracy = accuracy_score(y_test, y_pred)
print("\nModel Evaluation")
print("-------------------------------")
print("Accuracy:", accuracy)
# ------------------------------------------------
# 8. Confusion Matrix
# ------------------------------------------------
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix")
print("-------------------------------")
print(cm)
# ------------------------------------------------
# 9. Classification Report
# ------------------------------------------------
print("\nClassification Report")
print("-------------------------------")
print(classification_report(y_test, y_pred))
# ------------------------------------------------
# 10. Predict a new sample
# ------------------------------------------------
new_student = np.array([[7]])
prediction = model.predict(new_student)
probability = model.predict_proba(new_student)
print("\nPrediction for a student studying 7 hours")
print("-------------------------------")
if prediction[0] == 1:
    print("Predicted Class: PASS")
else:
    print("Predicted Class: FAIL")
print("Probability of FAIL:", probability[0][0])
print("Probability of PASS:", probability[0][1])
# ------------------------------------------------
```

```
# 11. Plot classification results
# ------------------------------------------------
plt.scatter(X, y, label="Actual Classes")
# Generate values for smooth sigmoid curve
X_plot = np.linspace(1, 15, 200).reshape(-1, 1)
probabilities = model.predict_proba(X_plot)[:, 1]
plt.plot(
            X_plot,
            probabilities,
            label="Logistic Regression"
        )
plt.xlabel("Study Hours")
plt.ylabel("Probability of Passing")
plt.title("Logistic Regression Classification")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT

```
Actual vs Predicted Values
--------------------------------
Actual: 1   Predicted: 1
Actual: 1   Predicted: 1
Actual: 0   Predicted: 0
Actual: 1   Predicted: 1
Actual: 0   Predicted: 0

Model Evaluation
--------------------------------
Accuracy: 1.0

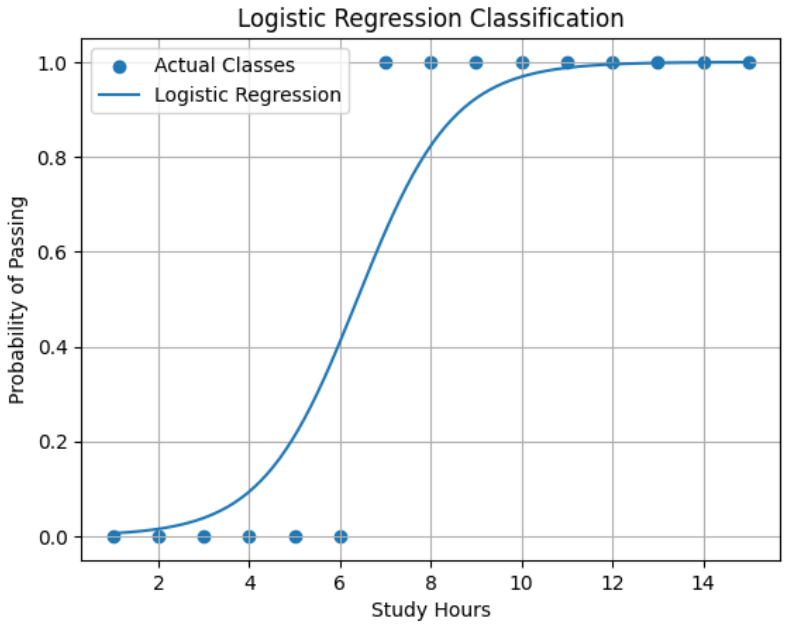
Confusion Matrix
--------------------------------
[[2 0]
 [0 3]]

Classification Report
--------------------------------
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         2
           1       1.00      1.00      1.00         3
    accuracy                           1.00         5
   macro avg       1.00      1.00      1.00         5
weighted avg       1.00      1.00      1.00         5


Prediction for a student studying 7 hours
--------------------------------
Predicted Class: PASS
```

```
Probability of FAIL: 0.35685226058201325
Probability of PASS: 0.6431477394179868
```



Prog -11

Python code to Demonstrate the use of Expectation Maximization based clustering algorithm

```
# Demonstration of Expectation-Maximization (EM)
# based Clustering using Gaussian Mixture Model
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture
# ------------------------------------------------
# 1. Create sample dataset
# ------------------------------------------------
X = np.array([
                [1.0, 2.0],
                [1.5, 1.8],
                [2.0, 2.5],
                [2.5, 3.0],
                [3.0, 2.8],

                [7.0, 8.0],
                [7.5, 8.5],
                [8.0, 9.0],
                [8.5, 8.0],
                [9.0, 9.0],

                [4.0, 5.0],
                [4.5, 5.5],
                [5.0, 4.5],
                [5.5, 5.0],
                [6.0, 6.0]
            ])
# ------------------------------------------------
# 2. Create Gaussian Mixture Model
# ------------------------------------------------
number_of_clusters = 3
gmm = GaussianMixture(
                        n_components=number_of_clusters,
                        covariance_type='full',
                        random_state=42
                    )
# ------------------------------------------------
# 3. Train the model using EM algorithm
# ------------------------------------------------
gmm.fit(X)
# ------------------------------------------------
# 4. Predict cluster labels
# ------------------------------------------------
```

```
labels = gmm.predict(X)
# ------------------------------------------------
# 5. Display cluster assignments
# ------------------------------------------------
print("EM-Based Clustering Results")
print("--------------------------------")
for i, label in enumerate(labels):
    print(
            f"Data Point {i + 1}: {X[i]} "
            f"-> Cluster {label + 1}"
        )
# ------------------------------------------------
# 6. Display cluster means
# ------------------------------------------------
print("\nCluster Centers (Means)")
print("--------------------------------")
for i, mean in enumerate(gmm.means_):
    print(f"Cluster {i + 1}: {mean}")
# ------------------------------------------------
# 7. Display covariance matrices
# ------------------------------------------------
print("\nCovariance Matrices")
print("--------------------------------")
for i, covariance in enumerate(gmm.covariances_):
    print(f"\nCluster {i + 1}:")
    print(covariance)
# ------------------------------------------------
# 8. Calculate membership probabilities
# ------------------------------------------------
probabilities = gmm.predict_proba(X)
print("\nCluster Membership Probabilities")
print("--------------------------------")
for i, probability in enumerate(probabilities):
    print(
            f"Data Point {i + 1}: "
            f"Cluster 1 = {probability[0]:.3f}, "
            f"Cluster 2 = {probability[1]:.3f}, "
            f"Cluster 3 = {probability[2]:.3f}"
        )
# ------------------------------------------------
# 9. Display model information
# ------------------------------------------------
print("\nModel Information")
print("--------------------------------")
print("Number of EM Iterations:", gmm.n_iter_)
print("Converged:", gmm.converged_)
```

```
print("Log-Likelihood:", gmm.lower_bound_)
# ------------------------------------------------
# 10. Predict a new data point
# ------------------------------------------------
new_point = np.array([[5.0, 5.2]])
new_cluster = gmm.predict(new_point)
new_probability = gmm.predict_proba(new_point)
print("\nPrediction for New Data Point")
print("--------------------------------")
print("New Point:", new_point[0])
print("Predicted Cluster:", new_cluster[0] + 1)
print("Membership Probabilities:", new_probability[0])
# ------------------------------------------------
# 11. Visualize the clusters
# ------------------------------------------------
plt.figure(figsize=(8, 6))
for cluster in range(number_of_clusters):
    points = X[labels == cluster]
    plt.scatter(
                    points[:, 0],
                    points[:, 1],
                    label=f"Cluster {cluster + 1}"
                )

# Plot cluster centers
plt.scatter(
                gmm.means_[:, 0],
                gmm.means_[:, 1],
                marker='X',
                s=200,
                label='Cluster Centers'
            )
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Expectation-Maximization Based Clustering")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT

```
EM-Based Clustering Results
--------------------------------
Data Point 1: [1. 2.] -> Cluster 3
Data Point 2: [1.5 1.8] -> Cluster 3
Data Point 3: [2.  2.5] -> Cluster 3
Data Point 4: [2.5 3. ] -> Cluster 3
Data Point 5: [3.  2.8] -> Cluster 3
Data Point 6: [7. 8.] -> Cluster 1
Data Point 7: [7.5 8.5] -> Cluster 1
Data Point 8: [8. 9.] -> Cluster 1
Data Point 9: [8.5 8. ] -> Cluster 1
Data Point 10: [9. 9.] -> Cluster 1
Data Point 11: [4. 5.] -> Cluster 2
Data Point 12: [4.5 5.5] -> Cluster 2
Data Point 13: [5.  4.5] -> Cluster 2
Data Point 14: [5.5 5. ] -> Cluster 2
Data Point 15: [6. 6.] -> Cluster 2

Cluster Centers (Means)
--------------------------------
Cluster 1: [7.99999976 8.4999996 ]
Cluster 2: [4.99998198 5.20000618]
Cluster 3: [2.00009795 2.42006804]

Covariance Matrices
--------------------------------

Cluster 1:
[[0.50000158 0.15000085]
 [0.15000085 0.20000211]]

Cluster 2:
[[0.50005622 0.15004575]
 [0.15004575 0.26003366]]

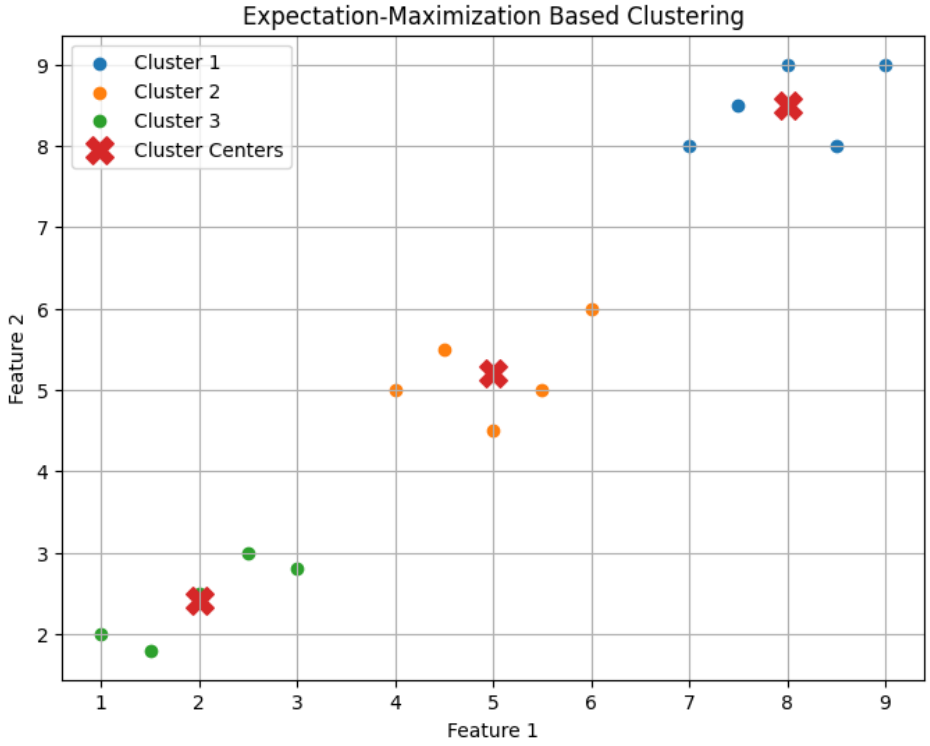
Cluster 3:
[[0.50029429 0.28020579]
 [0.28020579 0.20974314]]

Cluster Membership Probabilities
--------------------------------
Data Point 1: Cluster 1 = 0.000, Cluster 2 = 0.000, Cluster 3 = 1.000
Data Point 2: Cluster 1 = 0.000, Cluster 2 = 0.000, Cluster 3 = 1.000
Data Point 3: Cluster 1 = 0.000, Cluster 2 = 0.000, Cluster 3 = 1.000
Data Point 4: Cluster 1 = 0.000, Cluster 2 = 0.000, Cluster 3 = 1.000
Data Point 5: Cluster 1 = 0.000, Cluster 2 = 0.000, Cluster 3 = 1.000
Data Point 6: Cluster 1 = 1.000, Cluster 2 = 0.000, Cluster 3 = 0.000
Data Point 7: Cluster 1 = 1.000, Cluster 2 = 0.000, Cluster 3 = 0.000
Data Point 8: Cluster 1 = 1.000, Cluster 2 = 0.000, Cluster 3 = 0.000
Data Point 9: Cluster 1 = 1.000, Cluster 2 = 0.000, Cluster 3 = 0.000
Data Point 10: Cluster 1 = 1.000, Cluster 2 = 0.000, Cluster 3 = 0.000
Data Point 11: Cluster 1 = 0.000, Cluster 2 = 1.000, Cluster 3 = 0.000
Data Point 12: Cluster 1 = 0.000, Cluster 2 = 1.000, Cluster 3 = 0.000
Data Point 13: Cluster 1 = 0.000, Cluster 2 = 1.000, Cluster 3 = 0.000
```

```
Data Point 14: Cluster 1 = 0.000, Cluster 2 = 1.000, Cluster 3 = 0.000
Data Point 15: Cluster 1 = 0.000, Cluster 2 = 1.000, Cluster 3 = 0.000

Model Information
--------------------------------
Number of EM Iterations: 2
Converged: True
Log-Likelihood: -2.532786291890028

Prediction for New Data Point
--------------------------------
New Point: [5.  5.2]
Predicted Cluster: 2
Membership Probabilities: [1.23907171e-12 9.99999997e-01 2.66216225e-09]
```



**** END ****